CI/CD with Jenkins
Lab 1
Freestyle and Pipeline Job Basics

1 Basic Jenkins freestyle project with Shell commands

A Jenkins job (sometimes called project) is a user-configured description of work which Jenkins should perform as part of a standard DevOps workflow. Each job consists of various stages, which in turn contain a group of steps, where a step is a single fundamental task that tells Jenkins what to do.
Typical examples of these tasks include gathering dependencies, compiling, archiving or transforming code, and testing and deploying code in different environments.

Jenkins supports several types of jobs

a) freestyle projects
b) pipelines
c) multibranch pipelines
d) multi-configuration projects
e) folders
f) organization folders

The recommended job type for implementing practical CI/CD workflows in Jenkins is the pipeline.

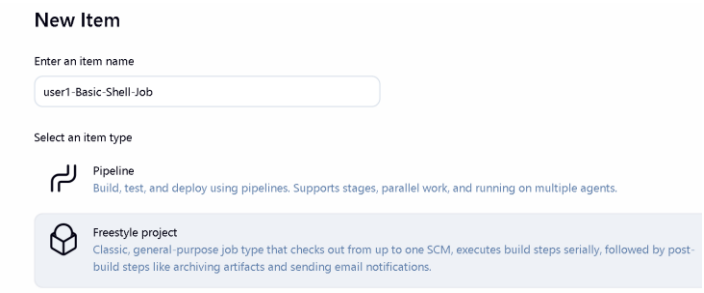
We will briefly look at configuring a simple freestyle projects, and focus primarily on using pipelines for the rest of the workshop.
The other kinds of jobs are specializations of pipelines used for very specific use case scenarios.

Ensure that you are logged into your assigned account on the Jenkins server / controller.

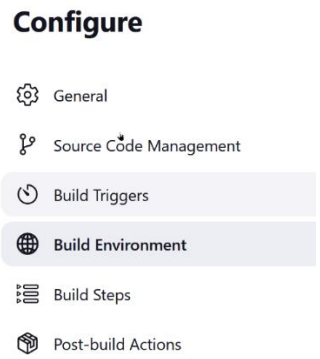
From the main Jenkins dashboard, select + New Item or Create a Job.

create a new Item and name it: `userx-Basic-Shell-Job`

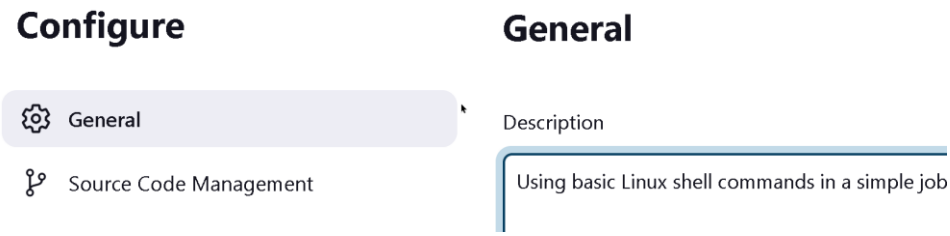
Make this a Freestyle Project and select Ok.



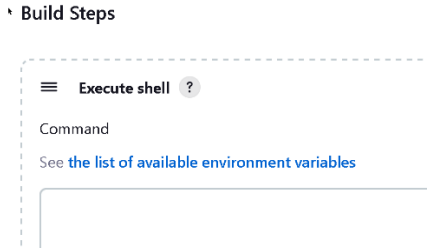
The configuration for this job is divided into several different stages / sections, which are also found in other projects. Each stage incorporates specific actions / steps to be performed. This step functionality is provided from the plugins that already installed on the main Jenkins controller, and any additional specialized functionality required will be obtained by adding more appropriate plugins.



Type some random info in the Description for this project in the General section.



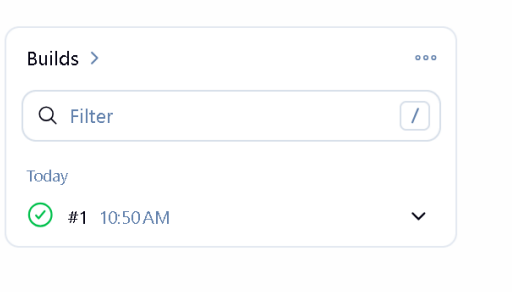
In the Build Steps section, choose Add Build step -> Execute Shell commands, and add in the following Linux shell commands:



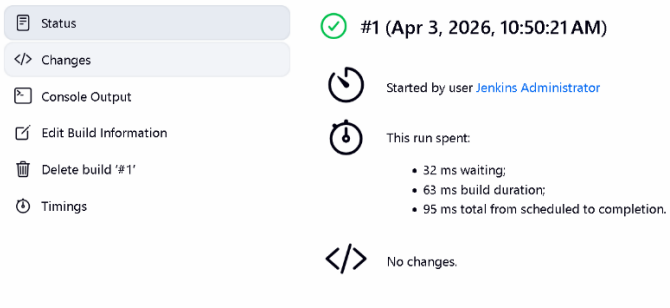
```
echo "Current active account is $(whoami)"
echo 'Creating a new directory if it does not exist and a new file'
mkdir -p awesome
cd awesome
NAME=Richard
echo "Hello, $NAME. The date is $(date)" > newfile
ls -l
pwd
echo "The content of newfile is "
cat newfile
```

Click Save.

Back at the main Job dashboard, click Build Now to run a build of the job. This should succeed and you should see a green circle next to a number (the particular build attempt) indicating success in the Build History.



Click on the Green Circle to get more detailed info on that particular build attempt.



The most important option you can explore from the navigation pane on the left is the Console Output, which is the log of all the activities that occurred during the build.

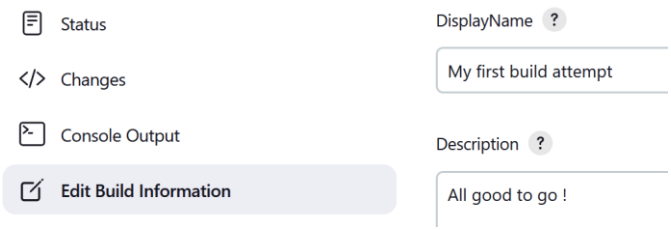
Some relevant points to note:

- The name of the logged in user (`Jenkins Main Administrator`) is shown
- The rest of the listing is the console output corresponding to the execution of the various shell commands. You can see that the active user account that is running the Jenkins main controller service is called `jenkins`, which is system user account (this is different from normal user accounts).
- The complete path of the workspace folder is shown. The default installation directory for Jenkins on a Linux machine is `/var/lib/jenkins`. Within this installation directory is a folder called `workspace`, and within `workspace` are all the workspace folders

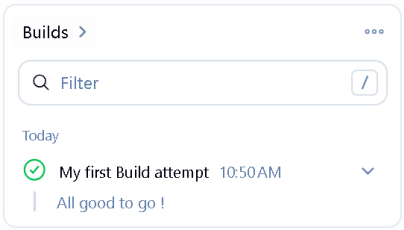
corresponding to all the jobs / items that you create in Jenkins. In this case, the complete path to the current workspace would be will be: `/var/lib/jenkins/workspace/userx-Basic-Shell-Job`. The final folder name will be the same as the job name, and all items / files related to that job will be placed within this workspace folder.

- All build-related activity for that job (creation of new directories, files, etc) will take place relative to the workspace folder for that job by default, unless the full path for another location is explicitly specified in one of the steps / tasks of the job.
- If you have any commands that manipulate directories or files, these directories or files will be relative to the workspace folder of that job. So, for e.g. if the `newfile` file that was created by one of the shell commands is within the `awesome` directory, which itself is within the top-level workspace folder.
- If you wish to access locations or files outside of the workspace folder, then remember to specify an absolute path: e.g. `/home/user1`
- However, remember that the Jenkins service is running as the `jenkins` user, so if you execute a shell command that attempt to access a directory / file / process that the `jenkins` user does not have permission to access, an access error will occur just as in the case for a standard Linux command.

Click on Edit Build Information to provide more random information on this particular build attempt, and then click Save.

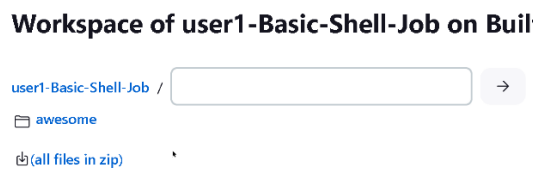


Return back to the main page for the job. Notice that the display name and description for this build attempt is now shown in the Build History.

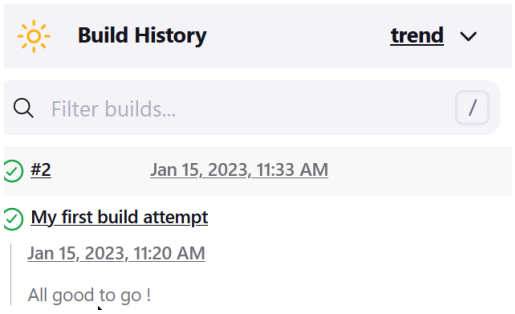


If you click on Workspace, you will be able to get a UI that allows you to navigate the workspace folder structure (which was created as part of the build process) as well as download the entire folder to our local machines in a zip format.

This would be useful here since the Jenkins service will typically be running on a remote server in a real life scenario. Go ahead and try that here.



Click Build Now again to perform another build attempt. This should succeed and the build attempt number is shown in the Build History.

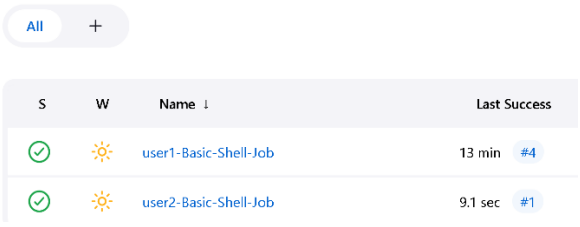


Repeat a build a few more times to see the build attempt list grow in the Build History. The Permalinks section in the middle provide links for you to navigate to different build attempts depending on their status (stable, successful, completed).

More info on build status concepts and terms:

So far for our very simple job, all the builds fall into this category.

If you now return to the main Dashboard, you will be able to see all the jobs that have been created on the Jenkins server by all logged in users up to this point of time (this means all the other participants in your assigned group, since they are all also working on the same Jenkins server).

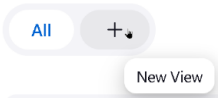


The various icons next to the job listing demonstrate the status of that job, and you can get more detailed info by just hovering your mouse over the particular icon.

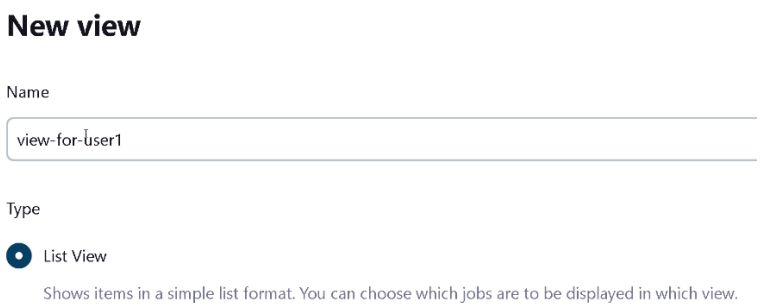
As the lab continues to progress the list of jobs available for inspection will become very long and it may be cumbersome to identify the jobs that you created. We can impose a filter on the list of jobs in the dashboard to simplify this.

2 Adding a filter to view the jobs in the dashboard

Select New View.

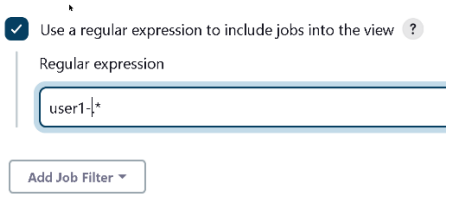


Give it the name: `view-for-userx`
And select the Type as List View.



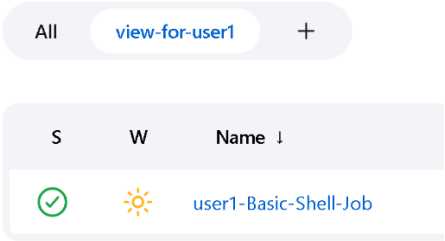
Select Create.

In the Jobs section, select Use a regular expression to include jobs into the view
And specify the regular expression: `userx-.*`



Click Ok.

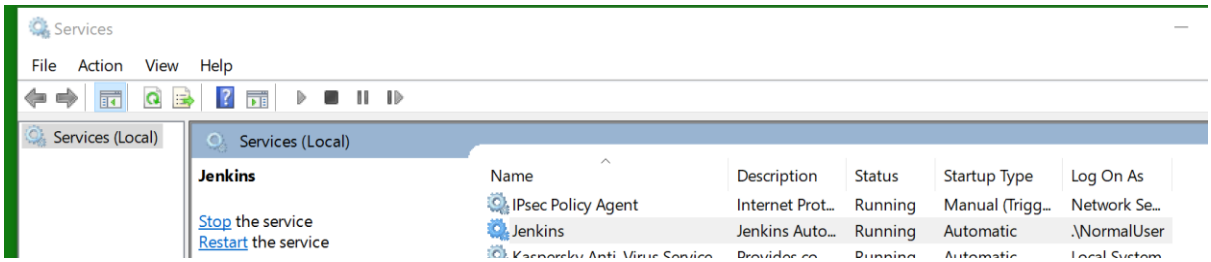
You should now only be able to see jobs that start with `userx-` in the main Dashboard.



You can always click on All to return back to view all available jobs in the main Dashboard.

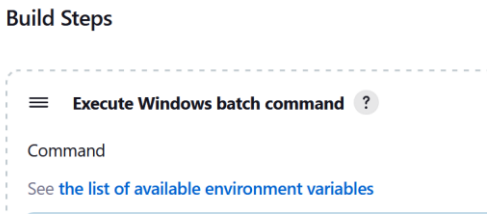
3 Basic Jenkins freestyle project on Windows (optional)

This is an optional exercise if you have managed to install Jenkins on your Windows machine successfully. Typically, it is installed on Windows as a service and will start on the default port of 8080. You can access it via the Services app.



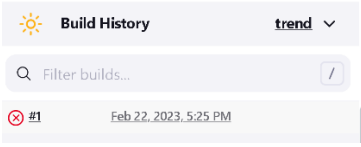
If Jenkins service is already running, you can open a browser tab to
http://localhost:8080
where you will see the main login page for the Jenkins UI, and you can login using the admin username / password combination that you registered during the installation process.

Repeat the process of creating a job that we did earlier, but this time in the Build Steps section, choose Add Build step -> Execute Windows Batch command



Now, see what happens if you attempt to reuse the list of shell commands that we entered previously in the lab on Linux shell command in the area for Windows batch command. Save and run a build on this job.

This time you will get an error for the build



And if you check the Console Output for this particular build, you will notice that errors are due to certain Linux shell commands not being recognized in Windows (`ls, pwd, cat`)

Now Configure this job again with proper DOS commands as shown below:

```
echo "Hi there from Jenkins"
echo "I love Jenkins ! Its the most awesome CI tool ever ... " >
hello.txt
echo "Showing contents of current job folder"
dir
type hello.txt
```

Save and build again. This time the build succeeds, because we are using DOS commands which can be interpreted and executed correctly by the Windows machine that the Jenkins server is running on. You can the Console Output to verify the actions that occur.

The complete list of DOS command line commands:
The list of most frequently used command line commands:

Some basic tutorials to working with the DOS command prompt:

Now if you repeat the process of taking these DOS commands and attempting to use them in the Execute Shell command section of the job that we created earlier on the Jenkins server running on a Linux machine, we will get similar errors for the same reason (DOS commands are not recognized and cannot be interpreted by the Linux Bash shell).

This demonstrates that there are certain aspects of a job configuration that are platform specific, although most aspects are platform independent. Later on, we will see when we are working with pipelines on how to execute generic file / directory operations in a general way through a Jenkins job regardless of the platform that Jenkins is running on.

4 Pipeline basics

We can consider a pipeline as a specialized job that is specifically meant to represent a model of a continuous delivery (CD) pipeline, performing the entire range of tasks that are typical for this pipeline. Just as is in the case of freestyle projects, these tasks are performed by installed plugins.

The functionality for a typical CI / CD pipeline can also be configured and achieved via a freestyle project. The main difference is that pipeline functionality is specified via a special kind of domain specific language (DSL) syntax, in what is known as the pipeline-as-code technique. This is opposed to freestyle projects whose functionality is specified through UI-based configuration options.

The definition of a pipeline is specified in a text file called a Jenkinsfile. There are two main ways to specify a Jenkinsfile (the syntax for both cases are identical)

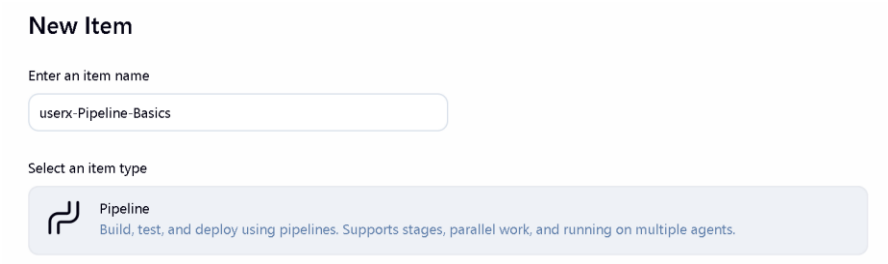
- written via the UI in the Script text area of the Pipeline section when a pipeline job is created.
- stored as a part of the project app files in SCM (GitHub, BitBucket, GitLab, etc), where it is versioned and reviewed just like any other source code artifacts (the more common approach)

There are two main forms of pipeline syntax: Declarative and Scripted. The primary differences between these two approaches are summarized here:

The recommendation for newbies and general users is to use the **declarative syntax approach**, which is what we will be focusing on for most of this workshop. Scripted syntax offers the most flexibility and power, but requires users be familiar with Groovy, the DSL variant of Java that is used here.

5 Pipeline basic structure

From the main Jenkins dashboard, create a new Item and name it: *userx*-Pipeline-Basics
Make this a Pipeline and select Ok.



You will be transitioned to the Configure section.

Notice now that there are no multiple stages / sections to configure the pipeline as you would in a normal freestyle project. This is because all the functionality of those stages / sections are going to be implemented as steps in the pipeline script.

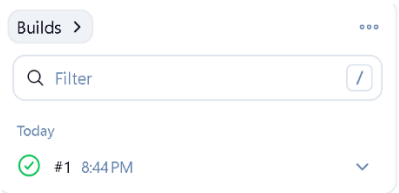
Select the Pipeline section.

Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `pipeline-basic-structure.txt`

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run your first build

Click on the completed build number to get more detailed build info.

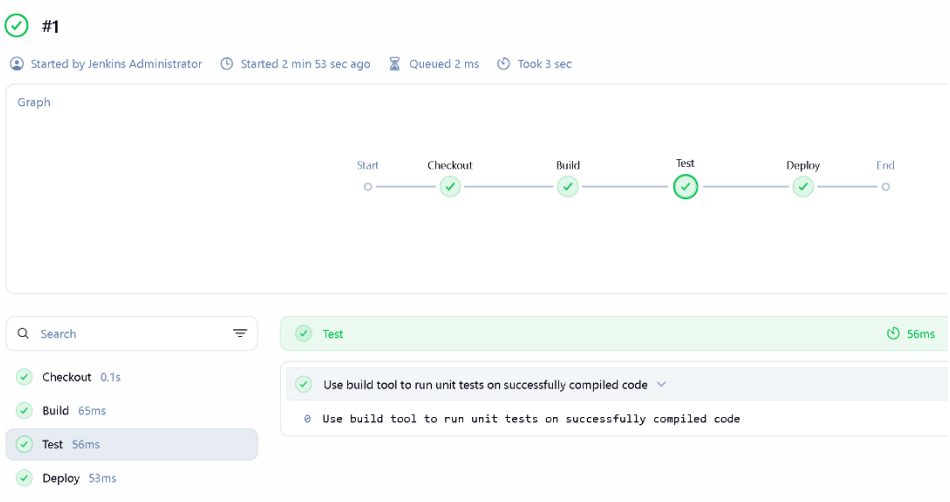


Key menu options to select from the main build page

- Console Output – view the console output from all stages of the pipeline job
- Pipeline overview – Allows you to view the output from each of the stages of the pipeline job
- Pipeline steps – Shows you the specific steps in each stage and how long it took to execute, as well as the corresponding output
- Workspaces – Allows you to navigate the workspace for that job after the completion of that current build. The workspace is the directory where Jenkins checks out the source code repository, runs the various commands for building, testing, etc in the various stages of the pipeline job, and creates build and test artifacts, as well as any other temporary files.

- Restart from stage – Allows you to run a new build by restarting from a particular stage of the current build.

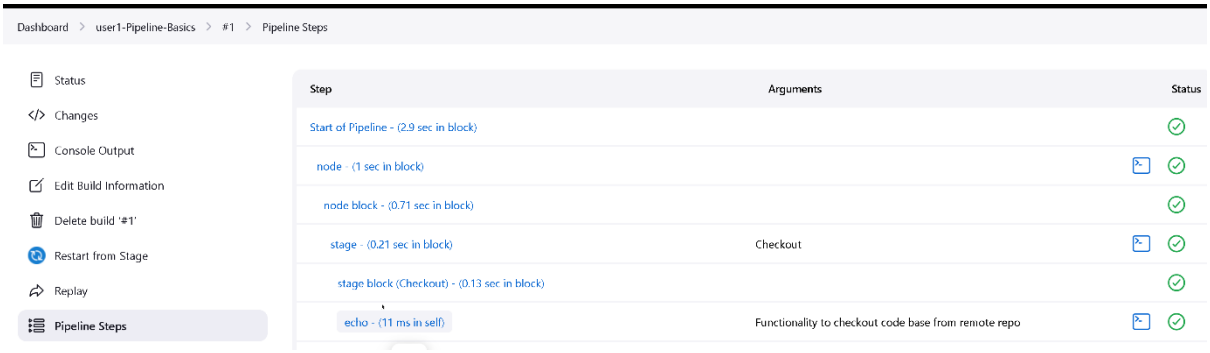
The Pipeline overview provides a detailed graph visualization of the execution of all the main stages in the pipeline.



The stages and steps in a pipeline are conceptually similar to the main stages that are available in the configuration of a freestyle project and the tasks available within each of those stages.

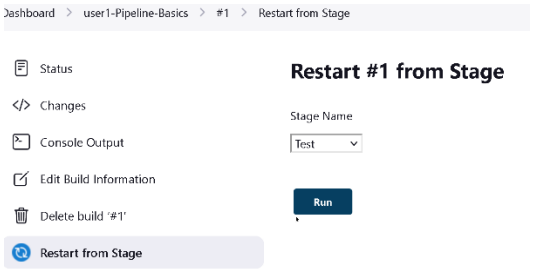
If you check the Console Output for a particular build, you will see the standard output (from the various `echo` statements in the script) marked as part of a Pipeline.

The Pipeline Steps provides detailed info on steps in each stage, where you can see the time taken to execute each stage, the status of execution (success / failure) and also to see the CLI output corresponding to that stage.

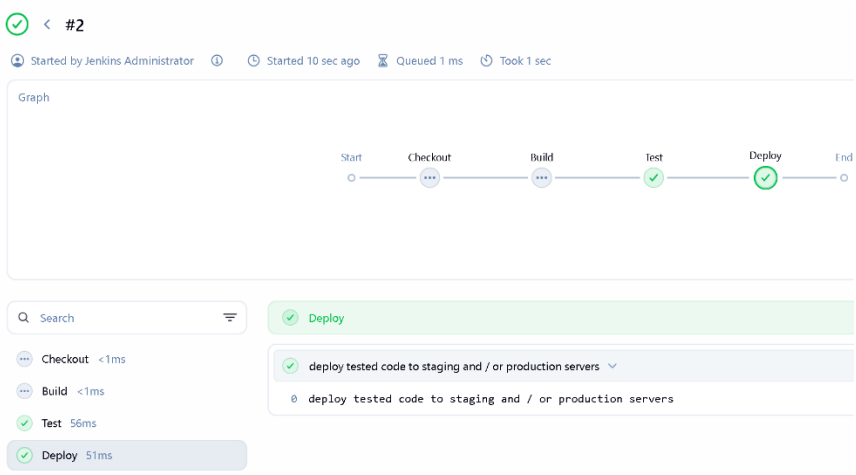


You can also choose to Replay (execute all the stages in the pipeline again), or to restart the pipeline from a given stage.

For e.g. you could choose to restart the pipeline again from the Test stage.



A new build will be executed and this time you will only see the stats related to the stages that you chose execution to commence from.



6 Pipeline: Nodes and Processes

In the current pipeline job, select Configure and select the Pipeline section

Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `pipeline-demo-sh.txt`

Click Save.

Back in the main page for the job, select Build Now from the left-hand menu to run another build and check the Console Output.
This pipeline job essentially replicates some of the functionality that we performed in an earlier freestyle project using Execute Shell Commands.

The `sh` step is one of the steps involved in the Pipeline: Nodes and process plugin, which is one of the core plugins that come with Jenkins (installed as part of the recommended plugins during initial installation).

Each step corresponds to functionality provided by the plugin and represents an atomic unit of action to be performed in a pipeline or freestyle project.

All the steps that you could use in a pipeline related to most plugins in the Jenkins ecosystem are accessible for reference from here. You can incorporate any of them directly into the `steps` section of the pipeline script, as long as you have their corresponding plugin installed:

The `bat, powershell, pwsh` and `sh` steps allow the execution of commands that are specific to those environments. Therefore, you have to ensure that they are only used on a Jenkins controller that is installed or has access to those environments.

As an optional exercise, if you have Jenkins installed on a Windows machine, create a pipeline job with the same name: `Pipeline-Basics`

Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `pipeline-demo-bat.txt`

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run a build and check the Console Output.

This pipeline job essentially replicates the functionality of the previous job that you created on Jenkins service on a Linux machine based on `pipeline-demo-sh`

Now, try and interchange the scripts, for e.g. copy and paste the script of `pipeline-demo-bat` into the job on Jenkins server in the Linux machine, or copy and paste the script of `pipeline-demo-sh` into the job on Jenkins server in the Windows machine. In both cases, running a build on these jobs will result in an error.

This is because the commands in those scripts are specific to the environment that Jenkins is installed in. For e.g. attempting to use `bat` on Jenkins running on a Linux machine will result in an error, and the same goes for `sh` on a Windows machine.

7 Pipeline: Platform independent steps

As we have just seen, some basic file / directory operations are platform specific (Windows / Linux). Often it is useful to create a pipeline script which can run independently of the platform that Jenkins is installed on.

From the main Jenkins dashboard, create a new Item and name it:
`userx``-Pipeline-Independent`
Make this a Pipeline and select Ok.

In the Configure area, select the Pipeline section

Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `pipeline-demo-independent.txt`

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run a build and check the Console Output.

The steps in the stage for this pipeline are taken from the Pipeline: Basic Steps plugin with the exception of the `dir` step which is from Pipeline: Nodes and process

Notice that it replicates some (not all) of the functionality from: `pipeline-demo-sh`

As an optional exercise, if you have Jenkins installed on a Windows machine, create a pipeline job with the same name: `Pipeline-Independent`

In the Configure area, select the Pipeline section

Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `pipeline-demo-independent.txt`

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run a build and check the Console Output.

In addition to the Pipeline: Basic Steps plugin, there are a few other plugins that allow you to perform basic file / directory related operations in a platform independent manner: